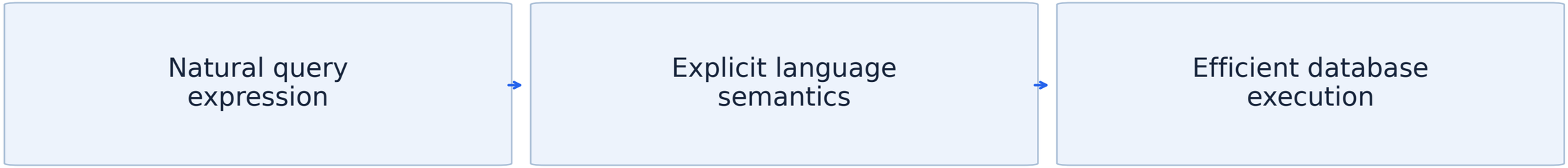
JacDB: Easier Queries, Efficient Execution

Group discussion • language design + compiler/runtime support • October 2026

Let programmers express database queries in application terms, and let the language implementation organize efficient execution.



Research framing: discussion with the project collaborators and user, 2026-10-21.

Two research questions, one programming model

Question	Desired outcome	Evidence
Q1: programming	Express traversal, filters and aggregation naturally	Query/modification tasks; errors; concepts; code
Q2: execution	Derive efficient access from that expression	Generated SQL; DB calls; transferred data; timing

The same program and workload should connect both evaluations.

Proposed research decomposition.

Make “comfortable to use” concrete

Express domain relationships without manually organizing per-object loads and joins.

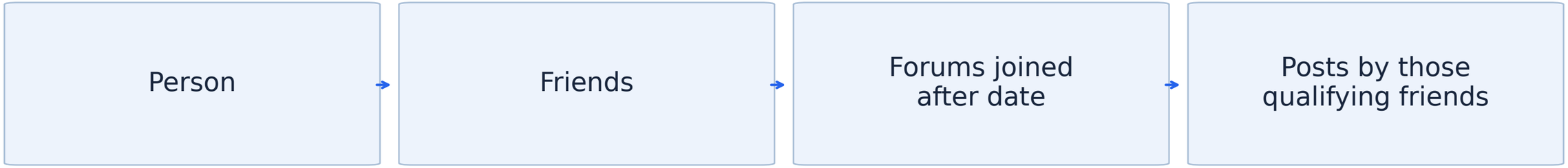
Reuse query logic across functions/abilities while preserving predictable execution boundaries.

Modify a predicate, aggregate or result shape with understandable types, errors and performance behavior.

Proposed criteria; Interdisciplinary PL Design, Fig. 1 / Table 1.

Running example: find relevant active forums

IC5: given a person and date, find friends’ recently joined forums and rank their qualifying posts.



Keep the (forum, friend) correlation → count matching posts → order → take 20.

An edge predicate, identity membership, aggregation and ranking all contribute to the answer.

Historical IC5 COMPILER_PLAN.md: query definition; benchmark/snb/jaseci-pg/app.jac.

Natural traversal leaves execution decisions open

Algorithm sketch adapted from the existing IC5 walker; illustrative pseudocode.

```
for friend in friends(person):
    for forum in joined_after(friend, date):
        qualifying[forum].add(friend)
for forum in qualifying:
    count posts whose author is in qualifying[forum]
return top_20(forums, by=(-count, forum_id))
```

The implementation chooses when to fetch, batch, join, aggregate and materialize.

Historical sources/session0904/ic5/COMPILER_PLAN.md, “The two plans”.

Actual Jac source: count qualifying authors

Verbatim inner-loop excerpt from the historical IC5 walker; enclosing forum loop supplies fo and foj.

```
cnt = 0;
for post in [fo->:ContainerOf:->[?:Post]] {
    creators = [post->:HasCreator:->[?:Person]];
    if creators and jid(creators[0]) in joined[foj] {
        cnt += 1;
    }
}
```

Question: can this remain the natural source while execution becomes a batched or relational aggregate?

sources/benchmark/snb/jaseci-pg/app.jac: ic5_new_forums, inner post loop.

What must remain predictable?

Source behavior	Compiler obligation
Same node reached twice	Preserve specified arrival / duplicate semantics
Count and rank	Preserve grouping, empty results and tie-breaking
Read, write, then read	Preserve visibility and transaction order
Permission checks / exceptions	Preserve authorized results and observable behavior

IC5 semantics; proposed JacDB semantic contract.

Prior work: language design already removes work

Collected backend/query-language references; detailed paper profiles follow in the appendix.

Work	Responsibility absorbed
Ur/Web	Typed persistence, encapsulation, transactions, retries, managed sends
Links / LINQ	Integrated queries, typed data access and object/result integration
Practical LINQ	Normalize composed query abstractions before SQL generation
Query Shredding	Flatten nested queries and reconstruct nested results

Ur/Web §§2–3; Links §§2,4; LINQ p.706; Practical LINQ §§2,5; Query Shredding §§4–5.

Prior work: several optimization paths overlap

Strategy	Collected references
Recover relational computation	QBS; Equivalent SQL; Froid; Compiling PL/SQL Away
Batch repeated access	Batched Bindings; Sloth
Move dependent work to the database	WeBridge; Pyxis
Choose profitable alternatives	COBRA

Jac’s research opportunity depends on its semantics, supported programs and measured programmer benefit.

Paper-lib: QBS, EE, Froid, PL/SQL Away, Batched Bindings, Sloth, WeBridge, Pyxis, COBRA.

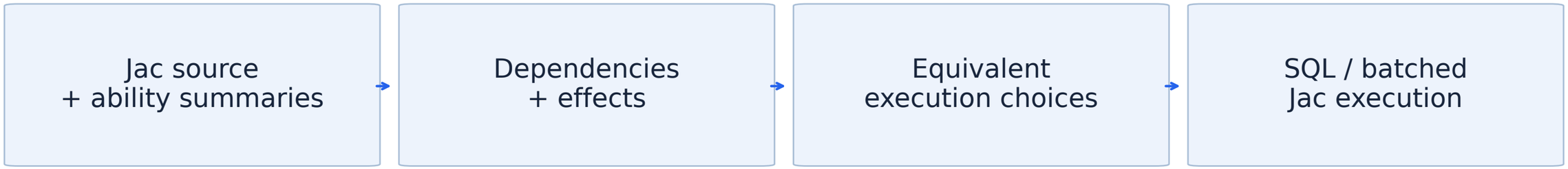
A language contract connects comfort to analysis

Programmer expresses	Language/compiler specifies
Typed nodes, edges and predicates	Persistent identity, relationships and field types
Traversal and reusable abilities	Arrival order, state flow and effect summaries
Aggregation and result construction	Bag/set/order semantics and materialization boundary
Atomic application operations	Transaction visibility and permitted effects

Proposed JacDB design; references: Ur/Web, Links, Practical LINQ.

Build a dependency representation before SQL

Hypothesis: a shared representation can replace isolated recognizers for a defined fragment.



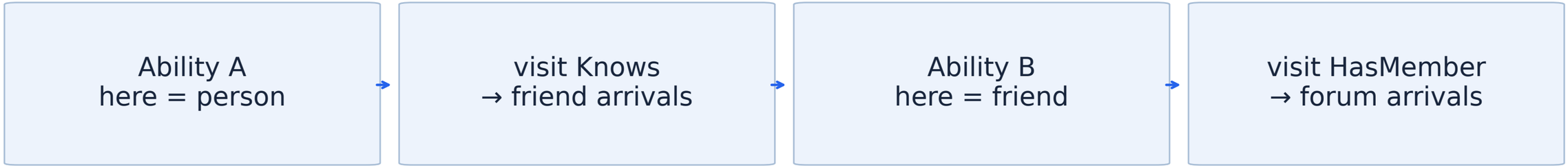
Track each value’s origin: parameter, node field, edge property, query or accumulator.

Track control, ordering and mutation dependencies alongside data dependencies.

Proposed design; related mechanisms: EE §§3–4; Froid §§3–5; WeBridge §§5–6.

Across abilities, track how here changes

A traversal result supplies the arrival node for the next supported ability.



Summaries carry arrival type, input lineage, walker-state reads/writes, emitted visits and effects.

Question to test: does this expose larger safe query regions than analysis within each ability?

Proposed analysis; existing Jac semantics supply traversal/ability structure.

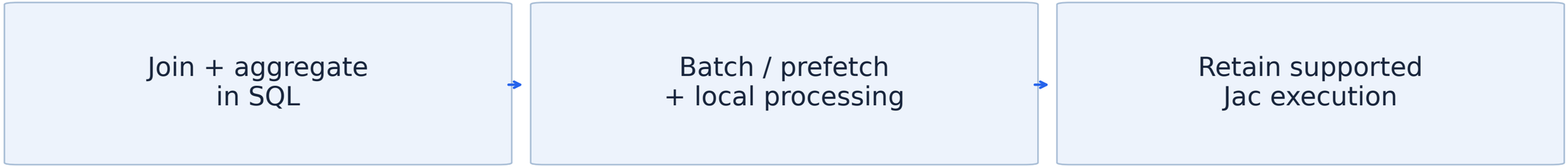
Eligibility and profitability are separate decisions

1. Is transformation legal?	2. Which legal plan is cheaper?
Known query semantics and lineage	Estimate cardinalities and result width
Preserved duplicates, ordering and state	Compare calls, transferred bytes and server work
Sound effect / alias / permission treatment	Include application work and memory use
Explicit boundary at unresolved behavior	Retain original execution as a candidate

EE §4.2; COBRA §§III–VI; Jac design questions.

Cost-based choices can preserve the same interface

The programmer’s query can have multiple equivalent implementations.

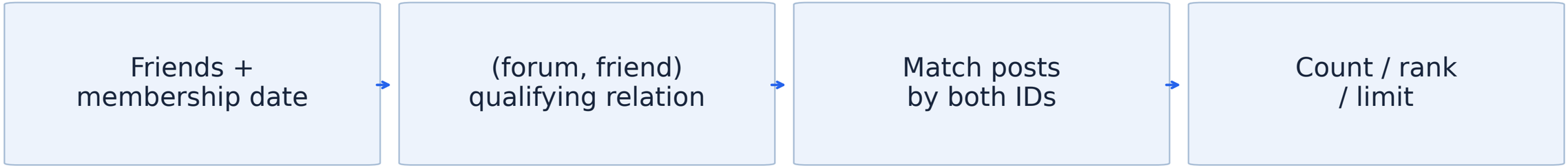


Selection depends on fan-out, repeated values, network conditions, database load and materialized data size.

Measure ranking accuracy and regressions when evaluating an automatic selector.

Apply the analysis to the same IC5 query

Recover the correlation and aggregate before choosing execution boundaries.



Batch frontiers to remove per-post calls; project only needed fields.

Recognize the correlated count and ranking; fetch selected forum fields.

Historical IC5 COMPILER_PLAN.md / REPORT.md; generalized dependency analysis is proposed.

Worked example: preserve correlation and zero counts

Teaching data, illustrating the required semantics; IDs are symbolic.

Forum	Qualifying friends	Stored posts	Qualifying count
10	Ann	Ann: 2; Bob: 3	2
20	Ann, Bob	Ann: 1; Bob: 2	3
30	Ann	None	0

Ranked result: (20, 3), (10, 2), (30, 0).

Use the matching friend for each forum; preserve qualifying forums with zero matching posts.

Adapted IC5 example; new illustrative data, independent of measured fixtures.

Existing mechanisms and the next abstraction

Established implementation	Proposed generalization
Typed lookup and graph predicates	An explicit supported query/traversal fragment
Loop prefetch and batched adjacency	Shared dependency/effect representation
Recognized nested-count lowering	Composable summaries across supported abilities
Projection, ranking and hydration choices	Costed alternatives with explanatory diagnostics

Historical IC5 REPORT.md / COMPILER_PLAN.md; current research proposal.

Existing evidence: database-related source lines

Each cell is total application LoC / database-related LoC under the historical classification.

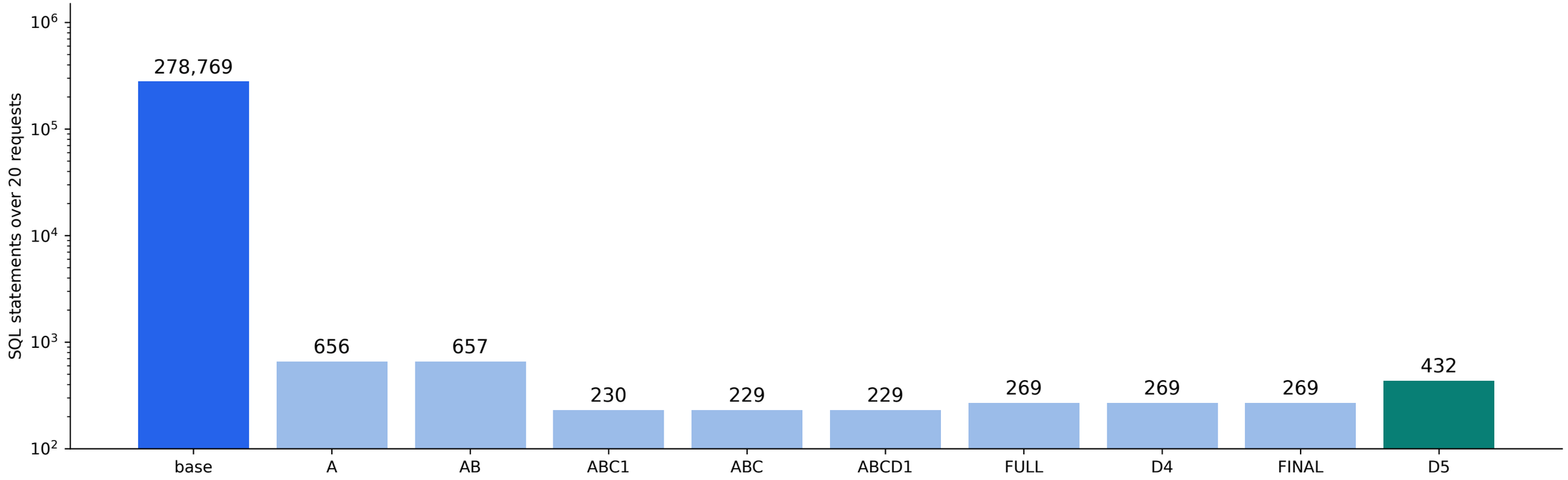
Application	Jac	PostgreSQL	SQLAlchemy
SNB	320 / 45	315 / 178	330 / 91
TPC-C	225 / 14	302 / 197	232 / 117
littleX	174 / 27	250 / 101	281 / 68

These implementations use fewer DB-related lines in Jac. Task completion and modification effort need direct measurements.

DBHarness 36f8b291; benchmark/burden/viewer_counts.csv; FORMAT_FAIRNESS.md.

Existing evidence: unchanged IC5, fewer SQL statements

20 parameter rows; identical walker; every rung passes result parity.



278,769 → 432 statements: 645.3× fewer, with unchanged application logic.

September 5 gates/ic5_*.json; SQL totals recomputed by this deck builder.

New measurements: define DB call time precisely

Primary RTT = per-request sum of complete database command/response durations.



Sum across all request-owned connections. The boundary includes server waits, communication, injected delay and compact-proxy overhead.

Application work between calls is measured outside this interval. SQL command counts come from the same requests.

webridge/non-snb/METHODS.md; RTT_MEASUREMENT_PLAN.md.

littleX feed: all compared source forms

20 tweets/user • zero added delay • 30 measured requests/cell • median summed DB RTT.

Implementation	DB RTT / request (ms)	Commands / request (mean)
Jac earlier runtime	13.635	18.67
Jac optimized	13.167	14.67
Java graph-access full	74.162	20.00
WeBridge full	Compiler failure	—
Java graph-access pruned	57.003	7.00
WeBridge pruned	Compiler failure	—
Java conventional SQL	9.516	5.00
WeBridge conventional	9.181	3.00

Jac: PostgreSQL 18. Java/WeBridge: MySQL 5.7. Compare optimization ratios within each engine.

webridge/non-snb/results.csv: littlex-feed-t20, delay_ms=0; METHODS.md.

Network delay makes remaining calls consequential

Same littleX feed workload; medians of summed DB-call time in milliseconds.

Implementation	+0 ms	+2 ms	+10 ms
Jac earlier runtime	13.635	59.895	227.773
Jac optimized	13.167	46.501	173.629
Java conventional SQL	9.516	18.082	59.959
WeBridge conventional	9.181	13.205	38.950

Per-request command counts and returned data explain the work boundary; the full trace records every exchange.

webridge/non-snb/results.csv: littlex-feed-t20, added response delays 0 / 2 / 10 ms.

Interpret the measurements at their actual boundaries

The historical IC5 ladder establishes a large reduction in generated SQL for unchanged code.

The new RTT matrix shows workload-specific gains and remaining calls. Earlier Jac already has batching; runtime versions are pinned.

Cross-engine absolute times describe complete measured implementations. Within-engine pairs isolate each optimization more directly.

METHODS.md; COMPLETION.json; historical-overview/verification.json.

Evaluate the programming claim with real tasks

Use the same features, data and correctness oracle across Jac, SQL and an ORM/query interface.

Task	What to observe
Write IC5-like query	Correctness, time, joins/batching written, errors
Change date filter / ranking	Concepts needed; files and semantic edits
Return nested forum/post results	Manual reconstruction and extra calls
Move logic into reusable abilities	Modularity, diagnostics and execution stability

Proposed study; Interdisciplinary PL Design Fig.1/Table1; ScalaLoci §8.

Evaluate the compiler claim on the same programs

Dimension	Experiment
Correctness	Empty results, duplicates, cycles, ACLs, writes, exceptions
Coverage	Inline vs helper vs cross-ability versions of the same query
Performance	Calls, DB RTT, bytes/rows, server time, separate application time
Profitability	Selectivity × fan-out × delay; compare all candidate plans

Proposed plan; EE / QBS / WeBridge / COBRA provide relevant baselines.

Group decisions: define the contribution to test

Choose the programmer-facing contract: which graph queries, composition and materialization rules?

Choose the core mechanism: dependency/effect summaries across supported traversal and ability boundaries.

Choose a decisive experiment: keep natural source structure, expand optimization coverage, measure effort and execution together.

Ur/Web: Typed persistence and atomic backend operations

Ur/Web: A Simple Model for Programming the Web

MECHANISM

Source table declarations, typed embedded SQL and modules connect persistent rows with language types. Remote calls execute atomically; managed channel sends are released at commit, and serialization failures trigger rollback/retry.

PAPER EVIDENCE

§§2.1–2.1.1, 3.1–3.2. §4 reports framework benchmarks, deployed applications and developer feedback. Developers explicitly specify SQL reads and writes.

CONNECTION TO JAC

Jac question: which query, transaction and effect-management responsibilities should the language absorb? Ur/Web is the closest collected backend language/database co-design reference.

JACDB / PRIOR-WORK APPENDIX

Links: Tables, query values and mutation in one language

Links: Web Programming Without Tiers

MECHANISM

A typed language spans browser, server and database. Mutable table handles are distinct from lists; a query view supports comprehensions, filters, ordering and take/drop that compile to SQL.

PAPER EVIDENCE

§2 Database subsection; §4 query compilation. Worked applications and translation rules explain the design. Table mutation is integrated with the source language.

CONNECTION TO JAC

Jac question: distinguish a persistent graph handle, a deferred query and a materialized result. Their types and execution boundaries affect both usability and optimization.

JACDB / PRIOR-WORK APPENDIX

LINQ: Composable queries plus object/relational integration

LINQ: Reconciling Objects, Relations and XML in the .NET Framework

MECHANISM

Standard query operators express traversal and filtering inside the host language. DLinq translates expression trees and object navigation into SQL, maps rows into objects and tracks updates.

PAPER EVIDENCE

Complete one-page system overview, p. 706, especially DLinq. It establishes an architecture; follow-up work supplies detailed semantics and evaluation.

CONNECTION TO JAC

Jac question: what work remains after query composition, mappings and change tracking are already provided? Compare specific programmer tasks against a capable ORM/query provider.

Practical LINQ: Composition needs a predictable compilation boundary

A Practical Theory of Language-Integrated Query

MECHANISM

Quotation retains query bodies; antiquotation composes predicates/functions. Normalization removes supported abstraction before SQL generation. A restricted typed fragment has a single-query guarantee.

PAPER EVIDENCE

§2; §5.3 Proposition 4; §9 evaluation. The formal core covers flat final results and excludes recursion, sorting, grouping and aggregation; practical coverage is broader.

CONNECTION TO JAC

Compiler background: define which Jac helpers retain query structure and which materialize results. Query-count guarantees require an explicit fragment; cost still needs separate evaluation.

Query Shredding: Nested results without a query per parent

Query Shredding: Efficient Relational Evaluation of Queries over Nested Multisets

MECHANISM

Normalize a query returning nested multisets, shred it into flat SQL queries, retain indexing information and stitch the results back into nested values.

PAPER EVIDENCE

Abstract, §1 and Fig. 1; mechanism in §§4–5; evaluation in §8. Query count depends on result/query structure rather than the number of parent records.

CONNECTION TO JAC

Jac question: can nested graph-shaped results retain a convenient interface while compilation handles batching and reconstruction? Preserve duplicates and specify ordering.

QBS: Recover relational intent from application loops

Optimizing Database-Backed Applications with Query Synthesis

MECHANISM

Synthesis produces loop invariants and postconditions in a theory of ordered relations; the resulting specifications translate into SQL for supported imperative fragments.

PAPER EVIDENCE

Abstract and §1. The program-to-query problem includes joins, aggregates and output order. Existing library coverage introduces the mechanism.

CONNECTION TO JAC

Jac question: which relational facts follow directly from typed graph operations, and which still require general program reasoning or synthesis? Test equivalent tasks and semantic edge cases.

Equivalent SQL: Variable lineage and folds expose relational computation

Extracting Equivalent SQL from Imperative Code in Database Applications

MECHANISM

D-IR tracks equivalent expressions and variable definitions; F-IR represents cursor loops as folds. Relational rewrites replace a selected computation with SQL and remove redundant code.

PAPER EVIDENCE

§§2–4, Fig. 6 and Appendix B. The maximum-score example preserves the initial accumulator on empty/all-negative input. Goals include fewer queries and less transferred data.

CONNECTION TO JAC

Jac question: use a dependency representation across supported statements/abilities, then derive aggregation. Preserve initial state, multiplicity, ordering, exceptions and effects.

Froid: Imperative scalar functions become relational regions

Froid: Optimization of Imperative Programs in a Relational Database

MECHANISM

UDF variables become columns; regions become relational expressions composed with APPLY. Inlining into the caller exposes the combined computation to the database optimizer.

PAPER EVIDENCE

§§3–5. The evaluated implementation disables prototype cursor-loop support (§4.2.1). §8.3 Fig. 13 reports about 3,300 versus nearly 5 million logical reads for its total-price example.

CONNECTION TO JAC

Jac question: can ability inputs/outputs become explicit region interfaces? Compare supported control flow and traversals. Froid and Equivalent SQL share composition and branching capabilities.

Compiling PL/SQL Away: Loops can target recursive SQL

Compiling PL/SQL Away

MECHANISM

Compile procedural functions into SQL subqueries; iteration becomes SQL-level recursion so the surrounding SQL query and function can execute together.

PAPER EVIDENCE

Abstract and §1 of the 2019 arXiv version. The motivating problem is procedural/SQL context switching inside the database.

CONNECTION TO JAC

Jac question: represent loop-carried state and termination explicitly when compiling cyclic traversal. Compare recursive SQL with retained application execution on the actual workload.

Batched Bindings: Repeated calls become a batch of parameter bindings

Rewriting Procedures for Batched Bindings

MECHANISM

Rewrite repeated query/procedure invocations into a batched call and rewrite the callee to accept a set of input parameters instead of one parameter at a time.

PAPER EVIDENCE

Abstract and first-page example. The target includes repeated query and stored-procedure/UDF calls; dependence and side-effect conditions determine legal rewrites.

CONNECTION TO JAC

Jac question: batch a frontier of traversal inputs across ability invocations and correlate every result with its input. Repeated arrivals and duplicates require defined behavior.

Sloth: Delay execution to discover larger batches

Sloth: Being Lazy is a Virtue (When Issuing Database Queries)

MECHANISM

Extended lazy evaluation exposes batching opportunities during execution, including across loops, branches and method boundaries, until a value is demanded.

PAPER EVIDENCE

Abstract and introduction. The paper evaluates more than 100 benchmarks from two applications and reports up to 3× page-load reduction under its conditions.

CONNECTION TO JAC

Jac question: compare static fusion with runtime deferral. Identify the exact demand that forces a query and whether a dependency can instead execute in the database.

WeBridge: Dependent access paths become stored procedures

WeBridge: Synthesizing Stored Procedures for Large-Scale Real-World Web Applications

MECHANISM

Record and replay hot paths using concolic execution, recover SQL control/data dependencies, synthesize procedures and intercept application database calls. Unseen paths use interactive execution.

PAPER EVIDENCE

§§1, 3–6. §10 explains complementary optimizations within generated database work. Procedure synthesis and relational rewriting can both reduce communication.

CONNECTION TO JAC

Jac question: can language semantics make cross-ability regions easier to identify and support? Compare matched workloads, generated work, fallback coverage and command counts.

Pyxis: Choose where application code executes

Automatic Partitioning of Database Applications

MECHANISM

Profile the application/server loads and analyze dependencies to partition code between application and database servers, reducing control transfers and transferred data.

PAPER EVIDENCE

Abstract and §1; evaluation uses TPC-C and TPC-W. The application/database placement decision incorporates resource load.

CONNECTION TO JAC

Jac question: after proving a region movable, choose its placement. Deployment support and database load are part of the execution decision.

COBRA: Choose among equivalent whole-program strategies

COBRA: A Framework for Cost-Based Rewriting of Database Applications

MECHANISM

A Region DAG retains equivalent imperative/relational alternatives. A cost-based search compares SQL joins, prefetch/local lookup and retained application computation.

PAPER EVIDENCE

§§II–VI. Fig. 12: at one million orders/73,000 customers, join vs prefetch is 6,047 vs 3,467 seconds on the slow network; 16 vs 12 seconds on the fast network.

CONNECTION TO JAC

Jac question: separate eligibility from profitability. Model round trips, row duplication/transfer, database execution, application work and available memory.

Eliom: Reusable behavior across explicit execution locations

Eliom: A Core ML Language for Tierless Web Programming

MECHANISM

OCaml annotations interleave client/server/shared code. Typed client fragments and cross-tier communication support components with behavior on both sides.

PAPER EVIDENCE

§2 examples; §§3–5 type system, execution and compilation. Placement and communication are explicit language choices.

CONNECTION TO JAC

Broader design lesson: modular source boundaries can span execution boundaries. For JacDB, test whether reusable query helpers/abilities remain analyzable when composed.

Hop: Package related logic while exposing execution levels

Hop, a Language for Programming the Web 2.0

MECHANISM

A two-level language packages application logic and GUI behavior together; function calls and event loops connect execution levels.

PAPER EVIDENCE

Abstract; §§2–4 constructs and execution; §6 implementation. Database access is supplied through libraries.

CONNECTION TO JAC

Broader design lesson: make execution boundaries understandable to programmers. A Jac query API should explain when construction becomes database interaction.

ScalaLoci: Placement types and generated communication

Distributed System Development with ScalaLoci

MECHANISM

Values are associated with peers using placement types. Architecture definitions and distributed reactive flows drive communication and data conversion.

PAPER EVIDENCE

§2 mechanisms; §8 case studies and runtime measurements. Table 1 counts LOC, callbacks, imperative state updates and remote accesses.

CONNECTION TO JAC

Evaluation lesson: measure specific responsibilities removed from source alongside runtime costs. Query execution and programmer effort need distinct evidence.

Multitier Modules: Separate modularity from deployment topology

Multitier Modules

MECHANISM

Modules encapsulate cross-peer functionality over abstract peer types, allowing composition and distribution to vary independently.

PAPER EVIDENCE

Abstract and §1; §5 case studies include algorithms, distributed data structures and Flink task distribution.

CONNECTION TO JAC

Broader design lesson: changing query placement should preserve a useful module interface. Study which Jac ability summaries allow analysis across reusable modules.

Orleans: Runtime responsibilities support a simpler object model

Orleans: Distributed Virtual Actors for Programmability and Scalability

MECHANISM

Virtual actors let the runtime handle activation, location and distributed resource management while application code uses logical actor identities.

PAPER EVIDENCE

§§2–3 programming/runtime model; §§4–5 applications and performance. This is the 2014 MSR technical report.

CONNECTION TO JAC

Broader design lesson: identify each responsibility moved into the runtime and its consistency contract. Apply that accounting to Jac object loading, identity and query execution.

Kappa: Compiler/runtime support absorbs infrastructure work

Kappa: A Programming Framework for Serverless Computing

MECHANISM

Checkpointing handles serverless timeouts, while concurrency mechanisms support parallel computation and coordination from a programming framework.

PAPER EVIDENCE

Abstract; §3 mechanisms; §5 evaluation; §6 programmer requirements. The abstraction comes with checkpoint/runtime constraints.

CONNECTION TO JAC

Broader design lesson: simplicity has a runtime cost and an effect boundary. Evaluate programmer obligations and execution overhead together.

PL design methods: Evaluate the programmer-facing claim directly

Interdisciplinary Programming Language Design

MECHANISM

Combine formal/performance methods with interviews, user experiments, case studies and iterative prototyping throughout language design.

PAPER EVIDENCE

Fig. 1, Table 1 and §4. The paper describes complementary design/evaluation methods, including human-centered approaches.

CONNECTION TO JAC

Jac evaluation: use representative query/modification tasks, comparable training and correctness tests. Measure completion, time, errors and required concepts alongside source metrics.